

Student: Colton Zachary

Overview

In this assignment, I implemented an AVL tree insertion algorithm using C++. An AVL tree is a self-balancing binary search tree where the difference in height between the left and right subtrees (balance factor) is kept between -1 and 1.

The goal was to ensure that after each insertion, the tree remains balanced by performing the appropriate rotations when necessary.

Implementation Details

The implementation was completed in avltree.h, specifically in the functions:

- Insert(...)
- RightBalanceAfterInsert(...)
- LeftBalanceAfterInsert(...)

These functions work together to maintain AVL balance during insertion.

Insert Function

The Insert function recursively places a value following BST rules. If the node is NULL, a new node is created. Values less than go left, greater values go right. Duplicate values are not inserted.

After insertion, the balance factor is recalculated. If it becomes greater than 1 or less than -1, rotations are performed.

Balancing Functions

RightBalanceAfterInsert handles right-heavy cases using rotations.

LeftBalanceAfterInsert handles left-heavy cases.

The four cases handled are:

- Left-Left (single right rotation)
- Right-Right (single left rotation)
- Left-Right (double rotation)
- Right-Left (double rotation)

Balance Factor

Balance factor = height(left subtree) - height(right subtree)

Valid values: -1, 0, 1

If it becomes ± 2 , rebalancing is required.

The program recalculates balance using `CalculateTreeBalance()`.

Testing and Results

The provided `main.cpp` was used for testing. Multiple insertions were performed.

The output shows that:

- BST ordering is preserved
- The tree remains balanced
- No skewed structures appear

This confirms correct AVL behavior.

```
/Users/coltonzachary/.zshrc:23: no such file or directory: /usr/libexec/java_home
(base) coltonzachary@Coltons-MacBook-Air-4 A4 % g++ main.cpp
./a.out
Tree 1:
Tree 1 Initial In Order Traversal:
57102030405060707980
Tree 1 Initial Level Order Traversal:
50307020406080107957
Removing 5, 10, and 30 from Tree 1
Tree 1 After Removal In Order Traversal:
720405060707980
Tree 1 After Removal Level Order Traversal:
504070206080779
Address of node containing 7: 0x101631d70 data from node containing 7: 7

Tree 2:
Tree 2 Initial In Order Traversal:
ABCDEFZ
Tree 2 Initial Level Order Traversal:
DBFACEZ
Removing F, C, and D from Tree 2
Tree 2 After Removal In Order Traversal:
ABEZ
Tree 2 After Removal Level Order Traversal:
EBZA
Data from node containing Z: Z

Int BSTREE
In order!
012345678910111213141516171819
Level order!
012345678910111213141516171819
Removing all items from the tree
In order after removal!

Level order after removal!

(base) coltonzachary@Coltons-MacBook-Air-4 A4 %
```

Conclusion

The AVL insertion algorithm was successfully implemented. The tree remains balanced after each insertion, ensuring $O(\log n)$ performance.

This demonstrates how AVL trees maintain efficiency compared to regular binary search trees.

